

Overlapping MoE Pre-Barriers with Data Transfer Using In-Network Computing

Abstract

Expert-parallel communication often waits for participating ranks to become ready before sending token data. At small batches, this pre-barrier can occupy a substantial part of Dispatch and Combine. Measurements of DeepEP V2 Direct on two H200 nodes show a pre-barrier duration near 15 μ s at EP8, accounting for about 20% of operator time at eight tokens per rank and about 5% at 128. In Qwen3-30B-A3B decode, pre-barriers account for 15.2–16.7% of the sampled cross-node GPU step.

We present RendezEP, an in-network computing design that overlaps readiness aggregation with data upload. A locally ready rank sends its readiness signal and begins uploading. INC buffers early data and forwards it once the required ranks and destination buffers are ready. Symmetric receive buffers provide known landing positions, while PFC handles staging pressure. We describe the protocol and analyze the resulting execution order. Under simultaneous readiness and unchanged forwarding throughput, early upload can hide one serialized one-way network traversal, less the design’s additional processing cost.

CCS Concepts: • Networks → Network protocol design; In-network processing; • Computer systems organization → Distributed architectures.

Keywords: expert parallelism, mixture-of-experts, in-network computing, synchronization, collective communication, datacenter networks

1 Introduction

Mixture-of-experts (MoE) inference routes token activations to selected experts and combines their outputs. Expert parallelism distributes those experts across GPU ranks, requiring Dispatch and Combine communication at every MoE layer. Communication libraries use GPU-initiated transfers, pre-allocated buffers, and topology-aware paths to move these data efficiently [3, 5].

Small batches make the coordination before transmission particularly important. A pre-barrier may prevent a rank from sending locally available tokens until it has received readiness signals from its peers. The token payload shrinks with batch size, but the readiness exchange still involves the same participating ranks.

Our measurements of DeepEP V2 Direct quantify this cost. With cross-node EP8, the pre-barrier lasts approximately 15 μ s across the measured small-batch configurations. It occupies about 20% of Dispatch and Combine time at eight

tokens per rank, falling to about 5% at 128. In Qwen3-30B-A3B decode, the accumulated pre-barrier intervals account for 15.2–16.7% of the sampled cross-node GPU step, compared with 7.9–9.3% within one node.

The opportunity comes from separating upload from destination delivery. A rank may have data ready to send even while another rank is still preparing its receive buffer. If the network can hold early data, the first rank can begin uploading during this wait. The destination remains protected until its receive buffer is writable.

We present RendezEP, which places readiness aggregation and bounded data staging at an INC node. Ranks announce local readiness and upload without waiting for peer signals to return. INC records the ready participants, buffers early data, and starts forwarding when all required readiness conditions hold. The receiving ranks use preallocated symmetric buffers, so forwarding does not wait for a global receive-count calculation.

This paper contributes a characterization of pre-barrier costs in H200 microbenchmarks and MoE inference, an early-upload protocol with explicit buffer-reuse and backpressure rules, and an operation-level analysis of readiness/data overlap. Following Swift’s separation of processing and network delay [9], the analysis identifies when early upload shortens the full communication operation. The benefit depends on rank readiness times, available staging, and the rate at which INC can forward buffered data.

2 Background and Motivation

2.1 Expert-Parallel Communication

An MoE layer selects up to $topk$ experts for each token. Under expert parallelism, Dispatch sends activations to the ranks hosting those experts; Combine returns their outputs to the token owner [3, 5]. Routing determines the destinations and traffic volume. The exchange also carries metadata needed to place tokens into expert input buffers.

Ranks communicate readiness, counts, and routing information through collective operations or point-to-point writes and polling. A readiness signal indicates that the resources needed for the round are available. Communication buffers provide known landing positions; the final arrangement of tokens by expert may depend on exchanged counts.

EP implementations organize these steps differently. NCCL EP HT gathers routing metadata before Dispatch. DeepEP V2 Direct performs a pre-barrier, then processes counts alongside data. NCCL EP LL writes into reserved slots with fine-grained signals [4, 12, 13]. These choices determine what must finish before a rank can send its first token.

2.2 Pre-Barriers in DeepEP V2 Direct

DeepEP V2 Direct uses GPU-initiated transfers and preallocated receive buffers [4]. In Dispatch, separate warps process token counts and layout metadata alongside token transmission. Combine reuses the routing information established during Dispatch.

Both operations perform a pre-barrier before these transfers begin. Each rank waits for the participating ranks to become ready and completes the required local synchronization. Consequently, even a rank with available input data must wait for peer readiness before sending its tokens.

This waiting cost is significant at small batches. In our cross-node EP8 measurements, the pre-barrier remains near 15 μ s, occupying about 20% of operator time at eight tokens per rank and 5% at 128. Section 5 reports the measurements. The opportunity is to begin moving locally ready tokens while readiness is still being collected.

2.3 Opportunities for In-Network Computing

In-network computing lets network devices process data while forwarding it, for example by summing values from several GPUs [15–17]. Such devices can also retain temporary state across packets. For pre-barriers, the useful state is which ranks are ready, together with space to hold data that arrives before forwarding is allowed.

A locally ready rank can upload while INC collects the remaining readiness signals. INC holds early tokens until the required receive buffers are writable, then forwards both buffered and arriving data.

Key insight. A rank can upload locally ready data before all ranks are ready, provided INC buffers the data until the required readiness conditions hold. This overlaps readiness aggregation with the first part of data transfer.

3 Design

RendezEP moves readiness aggregation to INC so that ranks can upload data while waiting for their peers.

3.1 Overview and Assumptions

Figure 1 contrasts the baseline’s wait-before-send sequence with early upload. INC maintains a bitmap of ready ranks and a bounded queue of early data for each communication round. A generation identifies the Dispatch or Combine operation, and a buffer epoch identifies a particular use of a receive buffer. All participating ranks and the traffic controlled by their pre-barrier must be covered by the INC node.

Receive buffers are allocated and registered symmetrically during initialization. INC obtains their layout, address mappings, sizes, and access permissions once. Each token carries routing information and an identifier that locate its slot in the destination’s preallocated buffer. INC can write tokens into these slots before all receive counts are known; the destination can arrange them by expert afterward.

A rank sends READY after its local input is available and the previous use of its receive buffer has finished. READY identifies the rank, generation, and buffer epoch. The node uses it to confirm that the buffer is writable for the current round. Deployment can be within or across nodes; any path that bypasses INC needs the corresponding readiness protection before it writes destination memory.

3.2 Early Upload and Readiness Aggregation

Each rank sends READY, then uploads locally available tokens with their routing information. Dispatch counts and other metadata follow the library’s normal processing schedule. READY does not wait for a complete count vector.

INC accepts one readiness announcement per rank and round, ignoring duplicate and stale announcements. Before all required ranks are ready, arriving data stays in network staging. After readiness is complete, INC forwards buffered and newly arriving tokens to their destinations at the available output rate. The forwarding condition is the same for every packet in the round: the required ranks have announced readiness and the destination buffer matches the announced epoch.

For Dispatch, the destination arranges received tokens into expert inputs using the routing and count metadata. For Combine, the same upload-and-forward sequence carries expert outputs back to their token owners for combination. Delivery acknowledgments and local processing determine when received data can be consumed and source buffers can be reused.

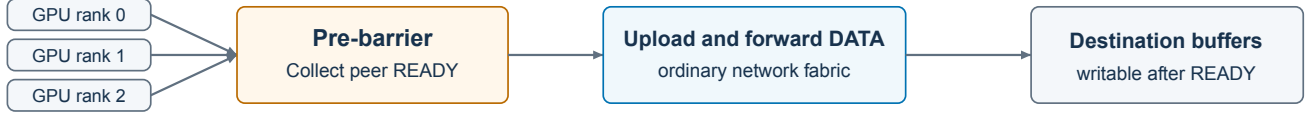
The source-to-INC transfer overlaps readiness collection. This can advance destination delivery when the output link can drain early data without introducing an equal or larger delay.

3.3 Buffer Management and Backpressure

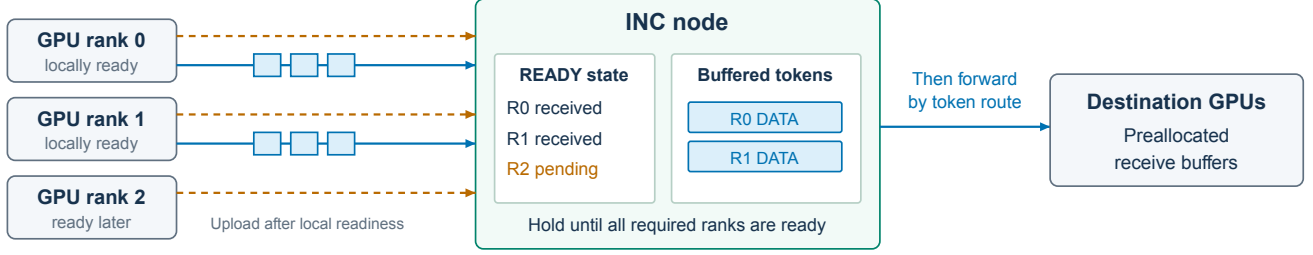
A receive buffer remains assigned to its round until the last computation reading it finishes. The next READY confirms that it can be reused. A source buffer remains allocated until sending hardware has finished reading it. Generation and epoch checks keep old messages from authorizing writes into a newer round. INC retires a round’s readiness state after its queued data has drained and all participants have advanced to the next round.

Network-buffer occupancy triggers PFC [8] before overflow, with headroom for data already in flight when a pause is sent. Upload resumes on the same path when capacity becomes available. READY uses a separate traffic class with reserved resources so that DATA pauses cannot block the signals needed to start forwarding. Required control metadata receives the same progress guarantee.

A Direct: wait, then upload



B INC: upload while collecting READY



Earlier upload; destination writes still wait for readiness.

Figure 1. Early upload overlaps readiness collection. The INC snapshot shows rank 2 still pending and data from ranks 0 and 1 buffered. Forwarding starts after all required ranks are ready and destination buffers are writable.

4 Latency Analysis

We model one complete Dispatch or Combine operation, from locally available inputs to completion on all participating ranks. Following Swift [9], we distinguish processing time from network traversal time and apply that distinction to the operation’s execution order.

4.1 Operation Latency

Consider a reference case in which ranks become ready together. Let L be the one-way network propagation and fixed forwarding delay. Let S be the time to transmit the operation’s full payload at its bottleneck rate, accounting for concurrent traffic from all ranks. Thus, S describes the collective data pipeline rather than a single flow.

Let H_{base} and H_{INC} denote processing time exposed on the critical path before and during delivery, including endpoint work and, for INC, readiness handling and buffering. Let C denote the subsequent delivery confirmation and local epilogue. Under the same payload rate and the same C , the reference schedules are

$$T_{\text{base}} = H_{\text{base}} + \underbrace{L}_{\text{pre-barrier}} + \underbrace{(S + L)}_{\text{data delivery}} + C, \quad (1)$$

$$T_{\text{INC}} = H_{\text{INC}} + (S + L) + C. \quad (2)$$

These are operation completion times. The terms describe successive work on the critical path, not sums of independently selected per-rank measurements.

4.2 How Early Upload Creates Overlap

In the baseline, a rank receives peer readiness signals before sending data. This exposes a control traversal ahead of

the payload. With INC, READY travels toward the network node while data follows it. If all required readiness signals reach the node before the corresponding data must be forwarded, token transmission proceeds without waiting for readiness signals to return to the source. Figure 2 illustrates this execution order.

For the reference schedule, the difference is

$$\Delta T = L + H_{\text{base}} - H_{\text{INC}}. \quad (3)$$

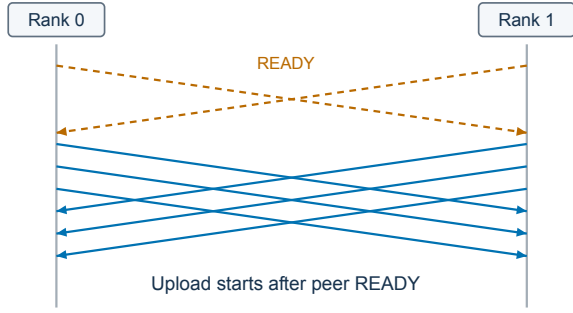
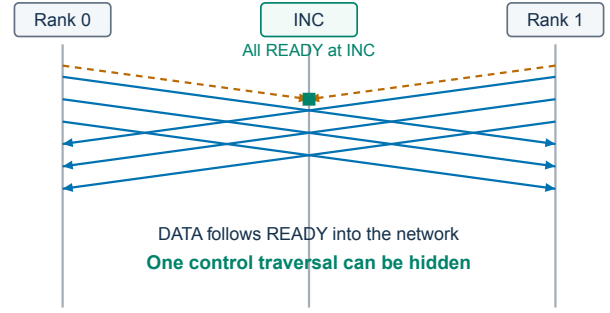
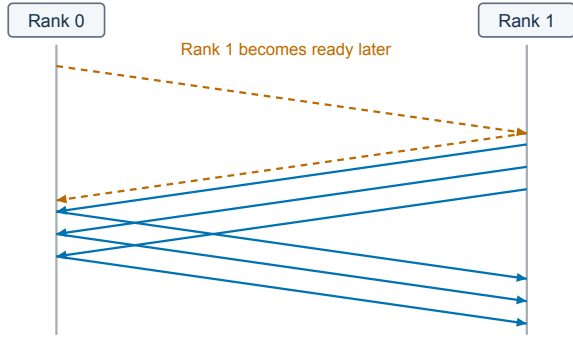
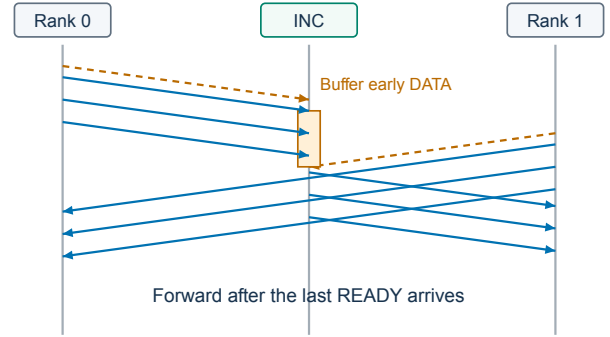
Early upload hides one serialized one-way traversal. Its net benefit also depends on the difference in exposed processing time. The reference assumes sufficient staging, no PFC pause, and an INC forwarding rate that sustains the baseline data pipeline. It also assumes that both designs require the same subsequent work C .

4.3 Unequal Readiness and Backpressure

When ranks become ready at different times, early ranks can fill INC staging while the remaining ranks prepare. Forwarding still waits for the required READY signals. The benefit depends on which data are ready at that point and whether the output link can forward them promptly.

A late rank may dominate the operation’s completion time, and accumulated data may queue at an INC output. PFC can further delay upload. In these cases, the comparison must follow the actual data-delivery and operation-finish times; the reference value L is not a fixed saving. Likewise, a pre-barrier measurement includes local work and peer waiting in addition to propagation.

For measurements, the operation completes when the last participating rank finishes. Earlier delivery at one destination contributes to an overall speedup only if it advances that

A Simultaneous readiness: Direct**B Simultaneous readiness: INC****C Unequal readiness: Direct****D Unequal readiness: INC**

Dashed: READY Solid: DATA Schematics show readiness and data delivery; spacing is illustrative.

Figure 2. Readiness and data transmission with simultaneous (top) and unequal (bottom) rank readiness. Dashed arrows are READY; solid arrows are DATA. INC holds early data until the last required READY arrives. The aligned reference can hide one control traversal; actual operation latency also depends on output queuing and subsequent work.

completion time. This connects the readiness-overlap mechanism to the operator and inference metrics in Section 5.

5 Evaluation

We characterize pre-barrier costs in Direct microbenchmarks and their contribution to MoE inference.

5.1 Experimental Setup and Methodology

Each node has eight H200 GPUs connected by NVSwitch and eight 400 GbE RoCE interfaces. Cross-node runs use GIN’s GDAKI backend, traffic class 162, PyTorch 2.13, CUDA 13, NCCL 2.31.2, and DeepEP V2 based on commit 01dc3aaa [4]. Direct is selected with Hybrid disabled; available local communication within Direct is retained.

Microbenchmarks fix $H = 7168$, $topk = 8$, 256 experts, BF16, and balanced uniform routing. T denotes tokens per rank. Each point has three independent process runs with 200 active iterations per run. Error bars show the standard deviation across run means.

For each iteration, we select the rank with the shortest pre-barrier interval and divide that duration by the same rank’s trace-on operator time. We average these ratios within each

run and then across runs. This statistic reduces the effect of rank arrival skew, while retaining any waiting experienced by the selected rank. Phase shares use paired trace-on measurements; trace-off operator timings are kept separately.

The EP8 share sweep covers $T = 8, 32, 64, 128$. The EP-size comparison uses the original measurements at $T = 8, 32, 96, 128$, supplemented by the later EP8 $T = 64$ batch. Configuration and per-run values accompany the data.

5.2 Pre-Barrier Cost

Figure 3 reports pre-barrier duration and operator share at EP8. Dispatch and Combine remain near $15 \mu s$ across the measured token counts. Their shares fall from 20.57% and 19.84% at $T = 8$ to 4.88% and 5.05% at $T = 128$. Small batches therefore expose a larger fraction of the operation to readiness coordination.

5.3 Dependence on EP Size

Table 1 summarizes pre-barrier duration with a separate constant for EP4, EP8, and EP16. All listed token counts participate in the estimate. The largest deviation of a per-token-count mean from its configuration’s constant is $0.16 \mu s$,

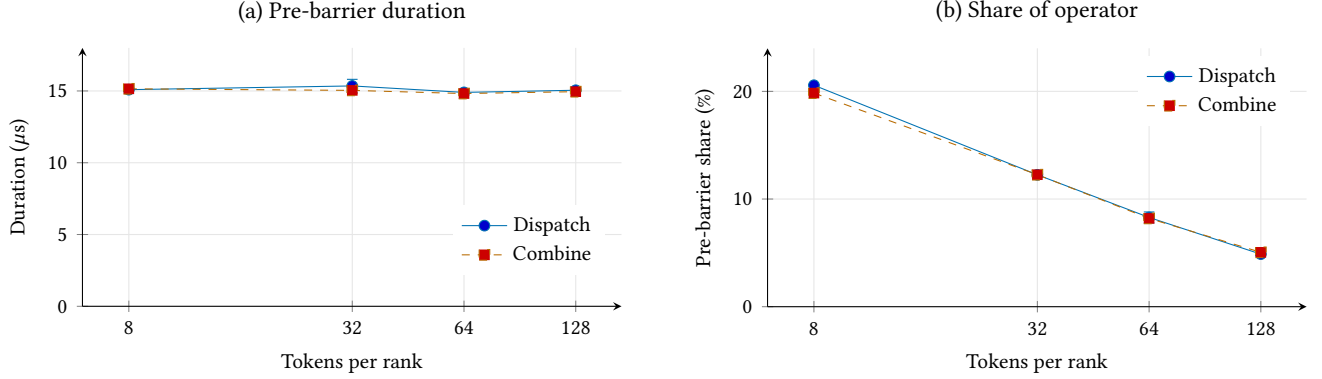


Figure 3. H200 cross-node EP8 Direct. Pre-barrier duration stays close to 15 μs while its operator share decreases. Both use the selected rank’s trace-on measurements; error bars are standard deviations across three run means. Lines connect measured points.

EP	Fitted T	Dispatch	Combine	Max. dev.
4	8/32/96/128	11.65	11.54	0.06
8	8/32/64/96/128	14.96	14.94	0.16
16	8/32/96/128	18.36	18.28	0.09

Table 1. Pre-barrier constants and maximum deviation of per- T run means, in μs . Each T has three process runs. EP8 includes a later $T = 64$ batch with different instrumentation.

supporting a nearly constant cost over the measured batch range.

The fitted duration rises from approximately 11.6 μs at EP4 to 18.3 μs at EP16. These configurations change GPUs per node, traffic destinations, and network concurrency together, so the observed variation reflects their combined effect.

5.4 Pre-Barriers in MoE Inference

We measure Qwen3-30B-A3B in BF16 with vLLM 0.27 and DeepEP V2 Direct. The model has $H = 2048$, 128 experts, and $\text{topk} = 8$. Runs use actual routing, $\text{TP}=1$, EP8, and requests with 128 input and 128 output tokens. We compare one node with eight ranks to two nodes with four ranks each, at concurrency 64 and 128.

The serving build includes tracing and an expert-padding correctness fix. Each sampled active rank has 48 Dispatch and 48 Combine trace entries per decode step. We sum its pre-barrier intervals, divide by that rank’s GPU-step time, and average over ranks and three client windows on the same server instance.

At concurrency 64, the pre-barrier share is 9.33% within one node and 16.72% across two nodes. At concurrency 128, the corresponding values are 7.94% and 15.19%. Thus, readiness coordination remains visible in the full decode step with real routing and model computation.

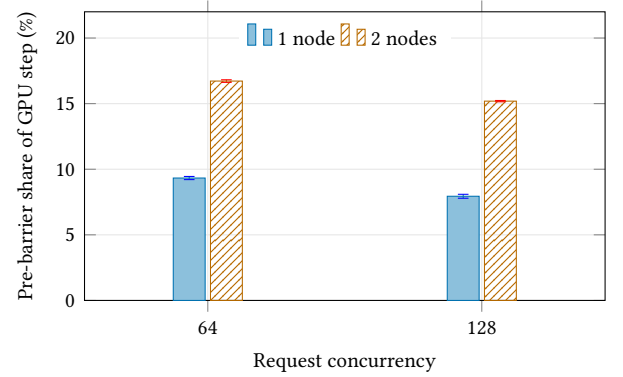


Figure 4. Qwen3-30B-A3B decode at fixed EP8. Pre-barrier intervals for Dispatch and Combine are paired with the same rank’s GPU step. Error bars describe three client windows, not server restarts.

These percentages describe sampled GPU steps. Unlike the microbenchmark statistic, they average over ranks rather than selecting the shortest interval. Together, the two experiments show both the operator-level cost and its place in inference.

6 Related Work

6.1 EP Communication

DeepEP [3] and NCCL EP [5] provide GPU-initiated communication paths for expert parallelism. Their metadata exchanges, preallocated buffers, and synchronization schedules determine when data can be sent. RendezEP uses network staging to begin upload while peer readiness is still being collected.

SwiftEP [10] optimizes buffer movement and GPU work, while COMET [18] overlaps communication with computation. Perseus [14] reorganizes PUT, fence, and signal processing in the transfer pipeline. UBEP [11] uses point-to-point signals and Data-as-Flag to reduce stage synchronization in scale-up EP. These approaches improve data movement or its coordination; early upload complements them when a readiness dependency precedes transmission.

6.2 In-Network Computation

SHARP [16] and SwitchML [15] use network computation for collective reduction. DySHARP [17] combines dynamic addressing, multicast, and reduction for MoE, while Multi-Write [2] uses multicast writes to reduce repeated upstream transfers. These systems demonstrate network processing and state management for distributed workloads. RendezEP applies temporary state and buffering to overlap pre-barrier coordination with upload.

6.3 Synchronization and Delay Analysis

EPIC [1] provides protocol and resource abstractions for Ethernet in-network computing. GPU-initiated networking [6] provides endpoint communication primitives. RendezEP combines these capabilities with readiness checks, symmetric receive buffers, and flow control.

Swift [9] separates processing and network contributions to delay. We apply that distinction to the execution order of an EP operation. FabricPerf [7] further motivates explicit timing boundaries; our pre-barrier measurements pair each interval with the corresponding operator or GPU step.

7 Conclusion

H200 measurements show that pre-barriers remain near 15 μ s for cross-node EP8 Direct over the measured small-batch range and account for 15.2–16.7% of sampled cross-node MoE decode time. RendezEP lets locally ready ranks upload while INC collects readiness signals, buffering early data until destination writes are safe. This changes the order of coordination and transmission without requiring receive-buffer addresses to be recomputed each round. The analysis relates the overlap to operation completion time and identifies staging capacity, readiness skew, and forwarding throughput as the main factors governing its benefit.

References

- [1] 2026. EPIC: Programmable Protocols and Resource Abstractions for Ethernet In-Network Computing. arXiv:2605.18683. <https://arxiv.org/abs/2605.18683>.
- [2] 2026. MultiWrite: Eliminating Redundant Upstream Traffic in Many-to-Many Communication. arXiv:2605.22428. <https://arxiv.org/abs/2605.22428>.
- [3] DeepSeek-AI. 2026. DeepEP: An Efficient Expert-Parallel Communication Library. <https://github.com/deepseek-ai/DeepEP>.
- [4] DeepSeek-AI. 2026. DeepEP V2 synchronization and kernel sources. Commit 01dc3aaac820. Files: common/comm.cuh, impls/hybrid_dispatch.cuh, impls/hybrid_combine.cuh. https://github.com/deepseek-ai/DeepEP/tree/01dc3aaac82068020353dce2c302e38153c0bfaa/deep_ep/include/deep_ep.
- [5] Amos Goldman, Nimrod Boker, Maayan Sheraizin, et al. 2026. NCCL EP: Towards a Unified Expert Parallel Communication API for NCCL. arXiv:2603.13606. <https://arxiv.org/abs/2603.13606>.
- [6] Khaled Hamidouche, John Bachan, Pak Markthub, Peter-Jan Gootzen, Elena Agostini, Sylvain Jauegy, Aamir Shafi, Georgios Theodorakis, and Manjunath Gorentla Venkata. 2025. GPU-Initiated Networking for NCCL. arXiv:2511.15076. <https://arxiv.org/abs/2511.15076>.
- [7] Songlin Huang and Chenshu Wu. 2026. FabricPerf: Measuring NIC-less Scale-Up Network through GPU Communication Kernel Profiling. Project repository. <https://github.com/open-neutrino/fabricperf>.
- [8] IEEE 802.1 Working Group. 2011. IEEE 802.1Qbb: Priority-based Flow Control. <https://1.ieee802.org/dcb/802-1qbb/>.
- [9] Gautam Kumar, Nandita Dukkkipati, Keon Jang, Hassan M. G. Wassel, Xian Wu, Behnam Montazeri, Yaogong Wang, Kevin Springborn, Christopher Alfeld, Michael Ryan, David Wetherall, and Amin Vahdat. 2020. Swift: Delay is Simple and Effective for Congestion Control in the Datacenter. In *ACM SIGCOMM*. 514–528. <https://doi.org/10.1145/3387514.3406591>.
- [10] Xingyi Li, Yadong Liu, Xiaojie Huang, et al. 2026. SwiftEP: Accelerating MoE Inference with Buffer Fusion and TMA Offloading. In *USENIX NSDI*. <https://www.usenix.org/conference/nsdi26/presentation/li-xingyi>.
- [11] Yipeng Liu, Chang Liu, Si Shen, et al. 2026. UBEP: Re-architecting Expert Parallelism Communication Library for Production Superpods. arXiv:2607.06202. <https://arxiv.org/abs/2607.06202>.
- [12] NVIDIA. 2026. NCCL EP HT device implementation. Commit 7b83616df3ae. https://github.com/NVIDIA/ncccl/blob/7b83616df3ae082a1f32bb74c27458bfe8153a13/contrib/ncccl_ep/device/hybrid_ep.cuh.
- [13] NVIDIA. 2026. NCCL EP LL device implementation. Commit 7b83616df3ae. https://github.com/NVIDIA/ncccl/blob/7b83616df3ae082a1f32bb74c27458bfe8153a13/contrib/ncccl_ep/device/ll_ep.cuh.
- [14] Byungsoo Oh and Rachee Singh. 2026. Eliminating Hidden Serialization in Multi-Node Megakernel Communication. arXiv:2605.00686. <https://arxiv.org/abs/2605.00686>.
- [15] Amedeo Sapio, Marco Canini, Chen-Yu Ho, Jacob Nelson, Panos Kalnis, Changhoon Kim, Arvind Krishnamurthy, Masoud Moshref, Dan Ports, and Peter Richtarik. 2021. Scaling Distributed Machine Learning with In-Network Aggregation. In *USENIX NSDI*. 785–808. <https://www.usenix.org/conference/nsdi21/presentation/sapio>.
- [16] Scot Schultz. 2024. Advancing Performance with NVIDIA SHARP In-Network Computing. <https://developer.nvidia.com/blog/advancing-performance-with-nvidia-sharp-in-network-computing>.
- [17] Qijun Zhang, Chen Zhang, Zhuoshan Zhou, et al. 2026. Accelerating MoE with Dynamic In-Switch Computing on Multi-GPUs. arXiv:2605.05607. <https://arxiv.org/abs/2605.05607>.
- [18] Shulai Zhang, Ningxin Zheng, Haibin Lin, et al. 2025. Comet: Fine-grained Computation-communication Overlapping for Mixture-of-Experts. arXiv:2502.19811. <https://arxiv.org/abs/2502.19811>.